

ePass3000GM 开发指南

1.0 版



版权所有©2012 EnterSafe

<http://www.EnterSafe.com>

EnterSafe 尽最大努力使这篇文档中的内容完善且正确。EnterSafe 对于由这篇文档导致的任何形式的直接或间接损失不负有责任。这篇文档的内容会跟随产品的升级而有所变化。

修改记录：

日期	版本	说明
2012 年 2 月	1.0	第一版

EnterSafe

软件开发协议

本《软件开发协议》（以下简称《协议》）是用户（个人或者单一机构团体）与 EnterSafe 之间有关随附本《协议》的 EnterSafe 软件产品的法律协议。本软件产品包括计算机软件，并且还可能包括电子文档、相关媒体和印刷材料（以下简称“软件产品”）。您一旦安装、复制或以其他方式使用本“软件产品”，即表示您同意接受本《协议》中的条款的约束。如果您不同意本《协议》中的条款，则您不得安装、复制或以其他方式使用本“软件产品”；您可以将本“软件产品”退还原购买处并取得全额退款。

1.软件产品使用许可

如果您遵守本协议的条款，EnterSafe 将授予您协议中所述的权利。

1.1 EnterSafe 授予您作为个人的、非独家性的许可证，仅供您为用于设计、开发及测试您的设计以及以任何 EnterSafe 产品一起运行的软件产品。您可在无数量限制的计算机上安装本“软件产品”的副本，但您必须是本“软件产品”的唯一使用者。如果您为一个机构团体，EnterSafe 授予您指定您组织内一位人员依以上所规定的方式使用本“软件产品”的权力。

1.2 EnterSafe 允许您将本软件合并或链接到您的计算机程序中，但本软件产品中被合并或链接的部分仍受本协议的约束。

1.3 您可以以存档为目的复制合理数量本软件产品的副本；但如果 Entersafe 通过公开声明或发布新闻的形式终止软件副本的使用，您必须马上遵守这个要求。

2.反向工程、反向编译、反汇编的限制

您不可以对本“软件产品”的部分或全部进行反向工程、反向编译或反汇编；尽管有这项限制，如果适用法律明示允许上述活动，则不在此限制范围。

3.禁止租借、传播或商业主办服务

您不可出租、租赁或出借本“软件产品”；或将本“软件产品”放在服务器上传播；或利用本“软件产品”提供商业主办服务。

4.责任限制和补救措施

无论任何原因（包括但不限于上述所有直接规定或一般性的合同规定或其它情况）发生的损害，EnterSafe 与其供应商在本协议条款下的所承担的全部责任以及全部损害的唯一补偿，不超出您购买本“软件产品”所支付的款额。

5.免责声明

在适用法律所允许的最大范围内，EnterSafe 或其供应商按“现有状况且包含所有错误”提供本“软件产品”或支持服务（如果有），并声明不承担所有其他明示、隐含或法定的担保、责任和条件。其中包括但不限于下列任何担保、责任或条件（如果有）：适销性、对于特定目的的适用性、可靠性或可用

性、回应的准确性或完整性、结果或工艺的精良性、无病毒以及无疏忽；还包括通过本“软件产品”或因使用本“软件产品”而提供或未提供支持服务或其他服务、信息、软件和相关内容。用户对本“软件产品”没有所有权、不受干扰的使用权、不受干扰的占有权、与说明一致或不侵权的任何保证或条件。

6.版权所有

EnterSafe 保留所有本《协议》中未明确授予您的权利，本“软件产品”受版权和其它知识产权法及相关条款的保护。EnterSafe 拥有本“软件产品”的所有权、版权和其他知识产权。

7.协议终止

本《协议》在终止前有效。若您违反本《协议》的任何条款，使用本“软件产品”的权利将自动终止。本“软件产品”必须被销毁或返回 EnterSafe。您可以销毁本“软件产品”及其所有副本以终止协议。但条款 2，3，4，5，6 将继续有效。

CE Attestation of Conformity



The equipment complies with the principal protection requirement of the EMC Directive (Directive 89/336/EEC relating to electromagnetic compatibility) based on a voluntary test.

This attestation applies only to the particular sample of the product and its technical documentation provided for testing and certification. The detailed test results and all standards used as well as the operation mode are listed in

Test standards: EN 55022/1998 EN 55024/1998

After preparation of the necessary technical documentation as well as the conformity declaration the CE marking as shown below can be affixed on the equipment as stipulated in Article 10.1 of the Directive. Other relevant Directives have to be observed.

FCC certificate of approval



This Device is conformance with Part 15 of the FCC Rules and Regulations for Information Technology Equipment.

USB



This equipment is USB based.

WEEE



Dispose in separate collection.



章节目录

第一章	开发综述	1
1.1	ePass3000GM 的应用开发接口	1
1.2	使用 MS Crypto API 接口开发 ePass3000GM 应用	1
1.2.1	信息隐藏	1
1.2.2	身份鉴别	2
1.2.3	完整性检测	2
1.2.4	CSP 与加密处理	3
1.2.5	CSP 上下文	3
1.2.6	CryptoAPI 体系架构	4
1.3	使用 PKCS#11 接口开发 ePass3000GM 应用	5
第二章	ePass3000GM 的 CSP 模块	6
2.1	ePass3000GM 的 CSP 模块描述	6
2.1.1	基本情况	6
2.1.2	特征	6
2.2	ePass3000GM 的 CSP 所支持的算法	7
2.3	ePass3000GM 所支持的函数实现	7
2.4	CSP 各函数所支持的参数	8
2.4.1	CPAcquireContext	8
2.4.2	CPGetProvParam	8
2.4.3	CPReleaseContext	9
2.4.4	CPSetProvParam	9
2.4.5	CPDeriveKey	9
2.4.6	CPDestroyKey	9
2.4.7	CPDuplicateKey	9
2.4.8	CPExportKey	10
2.4.9	CPGenKey	10
2.4.10	CPGenRandom	10
2.4.11	CPGetKeyParam	10
2.4.12	CPGetUserKey	11
2.4.13	CPImportKey	11
2.4.14	CPSetKeyParam	11
2.4.15	CPDecrypt	11
2.4.16	CPEncrypt	11
2.4.17	CPCreateHash	12
2.4.18	CPDestroyHash	12
2.4.19	CPDuplicateHash	12
2.4.20	CPGetHashParam	12
2.4.21	CPHashData	12
2.4.22	CPHashSessionKey	12
2.4.23	CPSetHashParam	12
2.4.24	CPSignHash	13
2.4.25	CPVerifySignature	13

2.5 函数调用说明13

2.5.1 基本说明13

2.5.2 开发示例13

第三章 ePass3000GM 的接口函数14

3.1.1 设备管理14

3.1.2 访问控制17

3.1.3 应用管理20

3.1.4 文件管理22

3.1.5 容器管理24

3.1.6 密码服务26

附录 缩略语及术语43

第一章 开发综述

本章讲述有关开发 ePass3000GM 应用程序的问题，涉及的内容包括 ePass3000GM 支持的各种开发接口和针对不同接口的开发方法。

- ePass3000GM 的应用开发接口
- 使用 MS CryptoAPI 接口开发 ePass3000GM 应用
- 使用《智能 IC 卡及智能密码钥匙密码应用接口规范》接口开发 ePass3000GM 应用

1.1 ePass3000GM 的应用开发接口

ePass3000GM 的应用程序开发是针对 PKI 应用的开发。ePass3000GM 提供了 PKCS#11 和 CSP for Microsoft CryptoAPI 2.0 两种主流的应用接口。这两个接口分别遵循 RSA 公司开发的 PKCS#11 标准和微软公司制定的 MS CryptoAPI 接口标准。由于这两个接口标准同时也被其他很多软硬件厂商所支持，因此，ePass3000GM 可以不需要二次开发就与符合这两种接口规范的应用集成使用。

1.2 使用 MS Crypto API 接口开发 ePass3000GM 应用

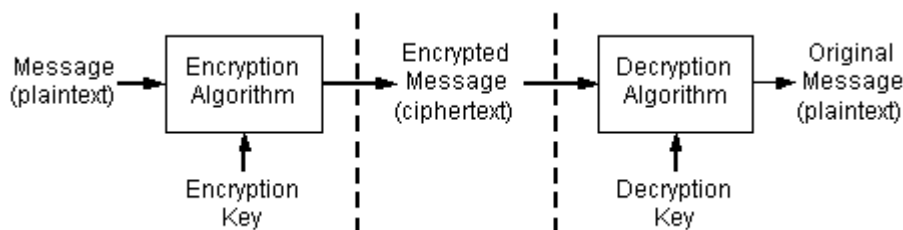
微软的 CryptoAPI 是 Win32 平台下为应用程序开发者提供的加密和安全的编程接口。CryptoAPI 函数集包含了基本的 ASN.1 编码、解码、散列、数据加密和解密、数字证书管理等重要的密码学应用功能。数据的加、解密支持对称算法和公开密钥算法两类算法。CryptoAPI 是所有微软的 Win32 应用程序及很多第三方厂商应用程序使用的加密接口，诸如 Internet Explorer 和 Outlook 等应用都是基于 CryptoAPI 开发的。

在不安全的网络上进行安全的数据传输涉及三个方面的要求：信息隐藏，身份鉴别和完整性检测。CryptoAPI 除了提供上述三个功能外还提供标准的 ASN.1 编码、解码、信息解密、数字证书和证书存储区的管理、证书信任列表、吊销列表和证书有效性检测等功能。

1.2.1 信息隐藏

信息隐藏的意义就是保证信息内容只能被特定的人获得。信息隐藏通常都要使用某种形式的密码学方法。数据加密算法能保证信息的安全隐藏和传输。数据加密算法将明文数据经过变化使其看上去像一组毫无规律的数据。在没有加密密钥的情况下，对于好的加密算法想从密文强制推导出明文是极其困难的。被加密的数据可以是任意 ASCII 码文本文件、数据库文件或其他任何需要进行安全传输的数据。在这里，“信息”这一术语用来表示任意一段数据，“明文”则指任意一段没有被加密的数据，“密文”则指任意一段被加密的数据。

被加密的数据可以通过不安全的介质或网络进行传输而不损害其安全性。之后，密文可被还原成明文，如下图所示：



数据加密和解密的概念非常简单，对数据加密的时候需要一个加密密钥，这里的密钥就相当于普通的门钥匙一样。解密的时候，需要使用一个解密密钥来解开数据。加解密密钥可以相同也可以不同。加密密钥必须小心保存，给其他用户时也必须通过安全的渠道传递。对解密密钥的访问权限必须小心控制，因为拥有解密密钥意味着可以解开所有用相应加密密钥加密的信息。

1.2.2 身份鉴别

安全通讯的前提是通讯的双方必须明确知道对方的身份。身份鉴别的任务就是鉴定一个用户或实体的真实身份。标识用户身份的文档通常被称为信任状或凭证。当兑现一张支票时，用户可以用身份证或驾照来表达自己的身份。身份证和驾照就是鉴别用户身份的有效凭证。护照是另外一个例子，海关官员通过护照来确定护照持有人的真实身份。海关官员信任发放护照的政府部门对持有人身份的鉴别。在上述两个例子中，用户身份的凭证存在于物理的文档中。

身份鉴别有时也用来判定接收到的数据就是被发送的数据。如果 A 向 B 发送了一段数据，B 需要鉴别这段数据就是 A 发出的，而不是其他人冒充 A 发出的。为了满足这类验证的要求，CryptoAPI 提供了数字签名和校验函数，来对信息进行鉴别。

因为在计算机网络上传输的数据与用户之间并没有物理的联系，因此对数据进行鉴别的凭证也必须能够在网络上进行传输。这种凭证必须由受信任的凭证发行机构发行。

数字证书也就是通常所说的证书就是这样一种凭证，是一种在计算机网络上进行身份验证的有效凭证。

数字证书是由一个被称为证书机构的被信任组织或实体颁发的凭证。它包含与证书对应的用户公钥以及其他一些记录证书主题和用户信息的数据。证书机构只有在验证了证书主题和证书对应的用户公钥的有效性后才会签发证书。

证书申请者和证书机构之间交互签发证书信息可以使用物理介质，比如软盘，进行传输。通常，这种信息交换都是在计算机网络上完成的。证书机构使用被信任的服务程序处理用户的请求和证书的签发工作。

1.2.3 完整性检测

任何通过不安全介质传输的信息都可能被意外或蓄意的修改。在现实世界中，印章就是用来提供和证明信息完整性的工具。例如一瓶阿司匹林使用不可还原的包装和完好的封印来证明它出厂后瓶里的药片没有变化。

同理，信息的接收者不但需要确定信息是由谁发送的，还需要确定自己收到的信息就是发送者发出

的信息，而没有任何变化。要建立数据的完整性检测机制，不仅需要发送信息本身，还需要发送用来校验数据的信息，这一信息通常被称作哈希值。数据和验证信息都可以与数字签名一起发送来证明其完整性。

1.2.4 CSP 与加密处理

Crypto API 函数使用“加密服务提供程序”(CSPs)完成数据加密、解密以及密钥的存储管理。所有的 CSPs 都是相互独立的模块。理论上，CSPs 应该是独立于特定的应用程序的，也就是说所有的应用程序都可以使用任何一个 CSP。但是，实际上有些应用程序只能与特定的 CSP 协作。CSP 与应用程序之间的关系就类似于 Windows GDI 模型。CSP 就类似于图形硬件驱动程序。

密钥存储的安全性完全取决于 CSP 的具体实现而与操作系统没有关系。这就使得应用程序无需修改就可以运行于多种安全环境之下。

应用程序与加密模块之间的访问必须受到严格的控制。只有这样才能保证应用的安全性和移植性。下面是三条应用原则：

- 应用程序不能直接访问密钥的内容。因为所有的密钥都是在 CSP 内部产生的，应用程序通过不透明的句柄对密钥进行访问。这就避免了应用程序和其关联的动态链接库泄漏密钥或使用不好的随机数产生密钥的可能。
- 应用程序不能指定加密操作的细节。CSP 接口允许应用程序选择进行加密或签名操作使用的算法类型，但是实际的操作完全由 CSP 内部进行控制。
- 应用程序不处理用户的信任凭证或其他身份鉴别数据。用户身份的鉴别是由 CSP 完成的。因此，对于未来可能出现的身份验证方式应用程序无需修改其身份验证模型。

最简单的 CSP 提供形式是由一个 Win32 动态链接库文件(DLL)和一个签名文件组成。必须提供正确的签名文件 CSP 才能被 CryptoAPI 识别和使用。CryptoAPI 对 CSP 的签名要经常进行周期性的检查，以防止 CSP 程序被篡改。

有些 CSP 模块通过本地 RPC 调用或硬件驱动程序在独立的地址空间中实现其关键的加密操作功能。将密钥和相关的加密操作放到具有独立地址空间的服务程序或硬件内可以保证密钥数据不被应用程序地址空间内的程序随意修改。

应用程序依靠某一 CSP 的特殊特性是很不明智的做法。例如，Microsoft Base Cryptographic Provider 现在提供 40 位的会话密钥和 512 位的公钥。应用程序应避免以此假设存储密钥数据所需的空间大小，因为一旦使用一个不同的 CSP 这一大小就可能不适用。好的应用程序应能与不同的 CSP 协同工作。

1.2.5 CSP 上下文

应用程序调用的第一个 CryptoAPI 函数必定是 CryptAcquireContext。这个函数返回包含指定了特定密钥容器的 CSP 操作句柄。密钥容器的选择可以特别指定也可以使用登录用户缺省的容器。CryptAcquireContext 也可用来创建新的密钥容器。

CSP 模块本身是具有名称和类型的。例如，Windows 操作系统缺省安装的 CSP 程序的名字是：Microsoft Base Cryptographic Provider，其类型是 PROV_RSA_FULL。每个 CSP 的名称都必须是不同的，

而类型可以相同。

当应用程序调用 `CryptAcquireContext` 函数获取一个 CSP 操作句柄时，可以指定 CSP 的类型和名字。如果指定了类型和名字则只有这两个属性匹配的 CSP 模块会被调用。调用成功后，函数返回 CSP 的操作句柄，之后应用程序就可以通过句柄访问 CSP 和 CSP 中的密钥容器了。

1.2.6 CryptoAPI 体系架构

CryptoAPI 体系架构由五个主要部分组成：

- 基本加密函数

用来连接和建立 CSP 操作句柄的函数。这组函数允许应用程序通过指定名称和类型来选择特定类型的 CSP 模块。

- 密钥生成函数

用来生成和保存加密密钥。其功能包括修改加密模式，初始化加密向量等加密特性。

- 密钥交换函数

用来交换和传输密钥。

- 数字证书编码与解码函数

这组函数用来加密和解密数据。其功能还包括数据散列计算的支持。

- 数字证书存储函数

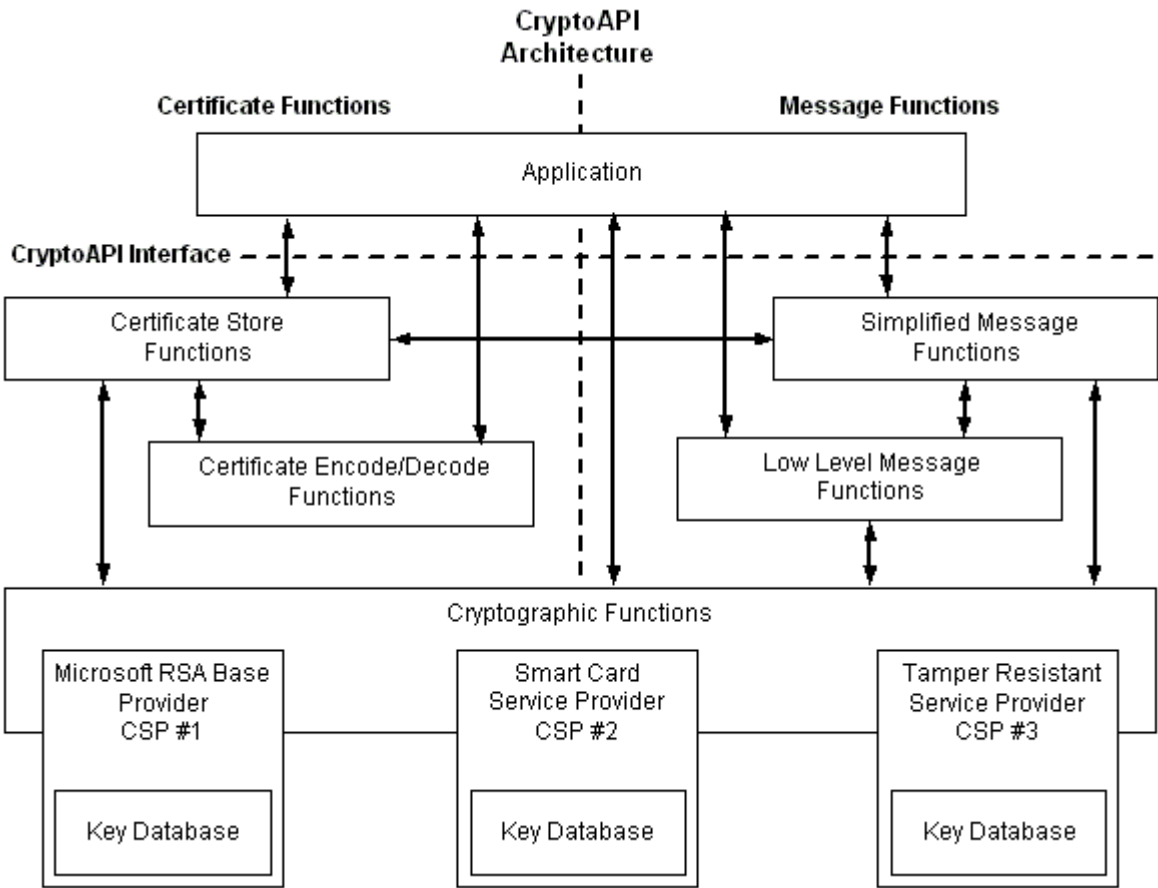
这组函数用来管理数字证书集合。

- 简化信息处理函数

这组函数用来加密和解密消息及数据，对消息和数据进行签名，验证消息及数据签名的有效性。

- 底层信息处理函数

这组函数是简化消息处理函数的实际实现函数。它提供了对消息进行各种操作的更为细致的控制。



每类函数的命名前缀都有约定，具体如下：

函数分类	前缀约定
基本加密函数	Crypt
数字证书编码与解码函数	Crypt
数字证书存储函数	Store
简化信息处理函数	Message
底层信息处理函数	Msg

1.3 使用《智能 IC 卡及智能密码钥匙密码应用接口规范》接口开发 ePass3000GM 应用

为了推进商用密码算法及应用的国产化，由国密商用密码管理局提出并制定了《智能 IC 卡及智能密码钥匙密码应用接口规范》，该规范规定了基于 PKI 密码体制的智能 IC 卡及智能密码钥匙密码应用接口，描述了密码应用接口的函数、数据类型、参数的定义和设备的安全要求。国家密码管理局网站 <http://www.oscca.gov.cn/>

第二章 ePass3000GM 的 CSP 模块

本章讲述 ePass3000GM 所支持 CryptoAPI 接口开发的相关情况，涉及的内容包括 ePass3000GM 的 CSP 接口名称及支持的函数和算法实现情况。

- ePass3000GM 的 CSP 模块描述
- ePass3000GM 的 CSP 函数的算法实现描述
- ePass3000GM 的 CSP 函数实现描述

2.1 ePass3000GM 的 CSP 模块描述

ePass3000GM 通过提供标准的 CSP 模块来实现与 Crypto API 应用程序的无缝集成。ePass3000GM 的 CSP 模块遵从微软的 Crypto Service Provider 编程规范编写，可以兼容现有的和将来的 Crypto API 应用。

2.1.1 基本情况

- 类型：PROV_RSA_FULL

这种类型的 CSP 支持数字签名和数据加解密，是一种通用类型的 CSP。所有的公钥运算都是使用 RSA 算法。

- 名称：EnterSafe ePass3000GM CSP v1.0
名称中指明了硬件类型为 ePass3000GM。

2.1.2 特征

ePass3000GM 的 CSP 具有以下几个特点：

- 提供了安全的 RSA 密钥对存储容器
- 提供多种分组加密算法和哈希算法
- 支持硬件实现的 RSA 运算，长度可达 2048 位
- 支持硬件随机数生成
- 支持多线程访问，多设备管理
- 支持多证书应用
- 和 PKCS#11 数据格式兼容
- 真正支持双证，即一个容器中可以有二个密钥对（AT_KEYEXCHANGE 和 AT_SIGNATURE）和对应证书
- 支持现有 Windows2000 及以上所有 Windows 32 位及 64 位平台
- 无缝兼容现有 Windows 平台应用，如 Office 加解密、Internet Explorer 的 WEB 网页登录和 SSL 网站登录、Outlook（Express）的安全电子邮件等。

2.2 ePass3000GM 的 CSP 所支持的算法

下表列出了 ePass3000GM 的 CSP 模块所支持的算法情况。

算法	默认长度 (bit)	最小长度 (bit)	最大长度 (bit)	用途类别
CALG_RC2	40	8	1024	加解密
CALG_RC4	40	8	2048	
CALG_DES	56	56	56	
CALG_3DES	192	192	192	
CALG_SSF33	128	128	128	
CALG_SCB2	128	128	128	
CALG_SHA1	160	160	160	散列
CALG_MD2	128	128	128	
CALG_MD5	128	128	128	
CALG_SSL3_SHAMD5	288	288	288	
CALG_RSA_SIGN 或 AT_SIGNATURE	1024	512	2048	签名验证
CALG_RSA_KEYX 或 AT_KEYEXCHANGE	1024	512	2048	加解密、 签名验证

2.3 ePass3000GM 所支持的函数实现

下表中列出了 CSP 接口各函数的支持和实现情况，“未实现”表示在 CSP 模块中有该接口，但是没有实现，“未支持”表示在 CSP 模块中没有该函数入口。

由于 ePass3000GM 的 CSP 类型为 PROV_RSA_FULL，不支持下表中所列出的标有“未支持”的函数是很正常的。标示为“未实现”的函数都返回 FALSE 并置 ErrorCode 为 E_NOTIMPL。这些函数是 CSP 接口，CryptoAPI 应用程序不需要直接调用这些接口。

名称	描述	支持情况
连接函数		
CPAcquireContext	为应用程序创建一个上下文	已实现
CPGetProvParam	返回 CSP 相关的信息	已实现
CPReleaseContext	释放 CPAcquireContext 创建的上下文	已实现
CPSetProvParam	设置 CSP 的参数操作	已实现
密钥生成和交换函数		
CPDeriveKey	从一个数据散列中生成一个会话密钥，它保证生成的密钥互不相同	已实现
CPDestroyKey	释放一个密钥句柄，释放后，句柄将无效，密钥将无法再被访问	已实现
CPDuplicateKey	创建密钥的一个拷贝	未支持

CPEExportKey	从 CSP 容器中导出密钥	已实现
CPGenKey	用来生成密钥或密钥对	已实现
CPGenRandom	使用随机数填充一个缓冲	已实现
CPGetKeyParam	用来得到加密操作密钥的属性	已实现
CPGetUserKey	用来获取 CSP 容器中的持久密钥对	已实现
CPImportKey	从一个 blob 中导入密钥到 CSP 容器中	已实现
CPSetKeyParam	设置密钥的属性	已实现
数据加密函数		
CPDecrypt	用来解密先前被加密的数据	已实现
CPEncrypt	用来加密明文	已实现
散列和数字签名函数		
CPCreateHash	初始化并散列输入数据	已实现
CPDestroyHash	删除一个散列对象句柄	已实现
CPDuplicateHash	创建一个散列对象的拷贝	未支持
CPGetHashParam	获取散列对象的计算结果	已实现
CPHashData	散列输入的数据	已实现
CPHashSessionKey	散列一个会话密钥而不向应用程序暴露密钥的值	未实现
CPSetHashParam	定制一个散列对象的属性	已实现
CPSignHash	签名一个散列对象	已实现
CPVerifySignature	校验一个数字签名	已实现

微软的 CSP 规范中还定义了 OffloadModExpo 这个 CSP 函数，这个函数目前不被 ePass3000GM 的 CSP 模块支持。

2.4 CSP 各函数所支持的参数

2.4.1 CPAcquireContext

——dwFlags

支持以下值：CRYPT_VERIFYCONTEXT、CRYPT_NEWKEYSET、CRYPT_DELETEKEYSET、CRYPT_SILENT；没有处理 CRYPT_MACHINE_KEYSET 的情况；

——pszContainer

根据 dwFlags 的不同值，pszContainer 可以为 NULL 和 “”，也可以为带有 Reader Name 的字符串，字符串长度不能超过 MAX_PATH。

2.4.2 CPGetProvParam

——dwParam

只支持以下值：PP_CONTAINER、PP_ENUMALGS、PP_ENUMALGS_EX、PP_ENUMCONTAINERS、PP_IMPTYPE、PP_NAME、PP_VERSION、PP_UNIQUE_CONTAINER、PP_PROVTYPE、

PP_SIG_KEYSIZE_INC 、 PP_KEYX_KEYSIZE_INC 、 PP_KEYSPEC ； 不 支 持 以 下 值 ：
PP_KEYSET_SEC_DESCR、PP_USE_HARDWARE_RNG 等。

——dwFlags

根据对微软 CSP 的分析，当 *dwParam* 的值为 PP_ENUMALGS、PP_ENUMALGS_EX 时，dwFlags 为 CRYPT_FIRST 开始枚举，为其它值时（包括 0 和 CRYPT_NEXT）枚举下一个。当 *dwParam* 的值为 PP_ENUMCONTAINERS 时，dwFlags 为 CRYPT_FIRST（1）或（CRYPT_FIRST|CRYPT_NEXT）（3）时开始枚举，为 0 或 CRYPT_NEXT（2）时枚举下一个。dwFlags 不支持 CRYPT_MACHINE_KEYSET。当 *dwParam* 的值为其他情况时，不检查 dwFlags 的值。

2.4.3 CPReleaseContext

——dwFlags

该值必须为 0。

2.4.4 CPSetProvParam

——dwParam

支持以下参数值： PP_KEYEXCHANGE_PIN、PP_SIGNATURE_PIN，此时当 pbData 为 NULL 时为退出登录。

不支持其他值。

——dwFlags

不检查该值。

2.4.5 CPDeriveKey

——AlgId

只支持以下算法：CALG_RC2、CALG_RC4、CALG_DES、CALG_3DES。

——dwFlags

对于以下情况报错：（CRYPT_CREATE_SALT | CRYPT_NO_SALT）、CRYPT_PREGEN、CRYPT_USER_PROTECTED，不支持其他值的情况。

2.4.6 CPDestroyKey

无需进一步说明。

2.4.7 CPDuplicateKey

不支持该接口。

2.4.8 CPExportKey

——dwBlobType

只支持 PUBLICKEYBLOB、SIMPLEBLOB，PLAINTEXTKEYBLOB 不支持 PRIVATEKEYBLOB、OPAQUEKEYBLOB 等情况。

——dwFlags

当 dwBlobType 为 PUBLICKEYBLOB、SIMPLEBLOB 时，dwFlags 必须为 0，其它情况忽略该参数的值。

2.4.9 CPGenKey

——AlgId

支持以下算法类型值：CALG_RSA_KEYX、CALG_RSA_SIGN、AT_KEYEXCHANGE、AT_SIGNATURE、CALG_DES、CALG_RC2、CALG_RC4、(CALG_3DES_112 下一版支持)、CALG_3DES。

——dwFlags

不支持 CSP 将返回错误信息：CRYPT_CREATE_SALT、CRYPT_NO_SALT、CRYPT_PGEN。其他情况将该参数的前两个字节的值作为要产生的密钥长度（前两个字节的值为 0 表示产生默认长度的密钥）。忽略该参数的后两个字节的值。

2.4.10 CPGenRandom

无需进一步说明

2.4.11 CPGetKeyParam

本函数只支持 CALG_RSA_KEYX、CALG_RSA_SIGN、AT_KEYEXCHANGE、AT_SIGNATURE、CALG_DES、CALG_RC2、CALG_RC4、CALG_3DES 等密钥类型。

——dwParam

对于 CALG_RSA_KEYX、CALG_RSA_SIGN、AT_KEYEXCHANGE、AT_SIGNATURE 等密钥类型，该参数的值可以为：KP_PERMISSIONS、KP_CERTIFICATE、KP_BLOCKLEN、KP_KEYLEN、KP_ALGID；对于 CALG_RC2 密钥类型，该参数的值可以为：KP_BLOCKLEN、KP_EFFECTIVE_KEYLEN、KP_KEYLEN、KP_ALGID、KP_SALT；对于 CALG_RC4 密钥类型，该参数的值可以为：KP_BLOCKLEN（返回值为 0）、KP_KEYLEN、KP_ALGID、KP_SALT；对于 CALG_3DES 和 CALG_DES 密钥类型，该参数的值可以为：KP_BLOCKLEN、KP_KEYLEN、KP_ALGID。

——dwFlags

必须为 0。

2.4.12 CPGetUserKey

——dwParam

支持以下参数的值：AT_KEYEXCHANGE、AT_SIGNATURE、(AT_KEYEXCHANGE | AT_SIGNATURE)

2.4.13 CPImportKey

——pbData

该 keyBlob 支持 SIMPLEBLOB、PUBLICKEYBLOB、PRIVATEKEYBLOB 这三种类型。

——dwFlags

忽略该参数。

2.4.14 CPSetKeyParam

——dwParam

KP_IV 参数值适用于 CALG_RC2、CALG_DES、CALG_3DES 类型密钥，KP_EFFECTIVE_KEYLEN 参数值适用于 CALG_RC2 类型密钥，KP_SALT 和 KP_SALT_EX 适用于 CALG_RC2 和 CALG_RC4 类型密钥，KP_CERTIFICATE 适用于 CALG_RSA_KEYX、CALG_RSA_SIGN、AT_KEYEXCHANGE、AT_SIGNATURE 等类型的密钥。

——dwFlags

必须为 0。

2.4.15 CPDecrypt

支持 CALG_RSA_KEYX、AT_KEYEXCHANGE、CALG_RC2、CALG_DES、CALG_3DES 和 CALG_RC4 等类型的密钥。

——dwFlags

必须为 0。

2.4.16 CPEncrypt

支持 CALG_RSA_KEYX、AT_KEYEXCHANGE、CALG_RC2、CALG_DES、CALG_3DES 和 CALG_RC4 等类型的密钥。

——dwFlags

必须为 0。

2.4.17 CPCreateHash

——AlgId

支持 CALG_MD2、CALG_MD5、CALG_SHA1 和 CALG_SSL3_SHAMD5 算法。

——dwFlags

必须为 0。

2.4.18 CPDestroyHash

无需进一步说明。

2.4.19 CPDuplicateHash

不支持该接口。

2.4.20 CPGetHashParam

——dwParam

支持 HP_ALGID、HP_HASHSIZE、HP_HASHVAL 等参数值。

——dwFlags

必须为 0。

2.4.21 CPHashData

——dwFlags

必须为 0，不支持 CRYPT_USERDATA 的参数值。

2.4.22 CPHashSessionKey

没有实现，返回 FALSE，并置 ErrorCode 为 E_NOTIMPL。

2.4.23 CPSetHashParam

——dwParam

仅支持 HP_HASHVAL 参数值。

——dwFlags

必须为 0。

2.4.24 CPSignHash

——sDescription

忽略该参数。

——dwFlags

支持 CRYPT_NOHASHOID 参数值，其它值忽略。

2.4.25 CPVerifySignature

——sDescription

忽略该参数。

——dwFlags

不支持任何值。

2.5 函数调用说明

2.5.1 基本说明

CPAcquireContext 函数是所有 CSP 函数中最先被调用的函数。上层应用通过调用这个函数来指定操作哪一个密钥容器。每一个密钥容器中同时只能保存同一种类型的一对 RSA 密钥对，和任意多个会话密钥。RSA 密钥对是可以持久保存的对象，而会话密钥则只在运行时存在。如果应用程序需要访问密钥容器中的 RSA 私钥，则 ePass3000GM 的 CSP 模块会要求验证用户的身份。如果应用程序希望避免弹出这个对话框则应当设置 CRYPT_SILENT 标志，但此时由于 ePass3000GM 不支持使用 CPSetProvParam 来设置用户身份识别信息，所以所有访问私钥和被保护数据的地方都会失败。

2.5.2 开发示例

开发人员可以在 SDK 包中的 Samples\CryptAPI 目录下找到使用 ePass3000GM 的 CryptAPI 接口的示例程序并可以编译调试。

有些示例程序您可能需要安装微软的 Platform SDK。

第三章 ePass3000GM 的接口函数

3.1.1 设备管理

3.1.1.1 概述

设备管理主要完成设备的插拔事件处理、枚举设备、连接设备、断开连接、获取设备状态、设置设备标签、获取设备信息、锁定设备、解锁设备和设备命令传输等操作。设备管理系列函数如表 16 所示：

表 16 设备管理系列函数

函数名称	功能
SKF_WaitForDevEvent	等待设备插拔事件
SKF_CancelWaitForDevEvent	取消等待设备插拔事件
SKF_EnumDev	枚举设备
SKF_ConnectDev	连接设备
SKF_DisconnectDev	断开连接
SKF_GetDevState	获取设备状态
SKF_SetLabel	设置设备标签
SKF_GetDevInfo	获取设备信息
SKF_LockDev	锁定设备
SKF_UnlockDev	解锁设备
SKF_Transmit	设备命令传输

3.1.1.2 等待设备插拔事件

函数原型	ULONG DEVAPI SKF_WaitForDevEvent(LPSTR szDevName, ULONG *pulDevNameLen, ULONG *pulEvent)
功能描述	该函数等待设备插入或者拔除事件。szDevName 返回发生事件的设备名称。
参数	szDevName [OUT] 发生事件的设备名称。 pulDevNameLen [IN/OUT] 输入/输出参数，当输入时表示缓冲区长度，输出时表示设备名称的有效长度，长度包含字符串结束符。 pulEvent [OUT] 事件类型。1 表示插入，2 表示拔出。
返回值	SAR_OK: 成功。 其他: 错误码。
备注	本函数为阻塞函数。

3.1.1.3 取消等待设备插拔事件

函数原型	ULONG DEVAPI SKF_CancelWaitForDevEvent()
------	--

功能描述	该函数取消等待设备插入或者拔除事件。
参数	
返回值	SAR_OK: 成功。 其他: 错误码。
备注	使本进程正在执行的 SKF_WaitForDevEvent 函数立即返回。

3.1.1.4枚举设备

函数原型	ULONG DEVAPI SKF_EnumDev(BOOL bPresent, LPSTR szNameList, ULONG *pulSize)
功能描述	获得当前系统中的设备列表。
参数	bPresent [IN] 为 TRUE 表示取当前设备状态为存在的设备列表。为 FALSE 表示取当前驱动支持的设备列表。 szNameList [OUT] 设备名称列表。如果该参数为 NULL，将由 pulSize 返回所需要的内存空间大小。每个设备的名称以单个 ‘\0’ 结束，以双 ‘\0’ 表示列表的结束。 pulSize [IN, OUT] 输入时表示设备名称列表的缓冲区长度，输出时表示 szNameList 所占用的空间大小。
返回值	SAR_OK: 成功。 其他: 错误码。

3.1.1.5连接设备

函数原型	ULONG DEVAPI SKF_ConnectDev (LPSTR szName, DEVHANDLE *phDev)
功能描述	通过设备名称连接设备，返回设备的句柄。
参数	szName [IN] 设备名称。 phDev [OUT] 返回设备操作句柄。
返回值	SAR_OK: 成功。 其他: 错误码。

3.1.1.6断开连接

函数原型	ULONG DEVAPI SKF_DisConnectDev (DEVHANDLE hDev)
功能描述	断开一个已经连接的设备，并释放句柄。
参数	hDev [IN] 连接设备时返回的设备句柄。
返回值	SAR_OK: 成功。 其他: 错误码。
备注	如果该设备已被锁定，函数应首先解锁该设备。断开连接操作并不影响设备的权限状态。

3.1.1.7获取设备状态

函数原型	ULONG DEVAPI SKF_GetDevState(LPSTR szDevName, ULONG *pulDevState)
------	---

功能描述	获取设备是否存在的状态。	
参数	szDevName	[IN] 设备名称。
	pulDevState	[OUT] 返回设备状态。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.1.8 设置设备标签

函数原型	ULONG DEVAPI SKF_SetLabel (DEVHANDLE hDev, LPSTR szLabel)	
功能描述	设置设备标签。	
参数	hDev	[IN] 连接设备时返回的设备句柄。
	szLabel	[IN] 设备标签字符串。该字符串应小于 32 字节。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.1.9 获取设备信息

函数原型	ULONG DEVAPI SKF_GetDevInfo (DEVHANDLE hDev, DEVINFO *pDevInfo)	
功能描述	获取设备的一些特征信息，包括设备标签、厂商信息、支持的算法等。	
参数	hDev	[IN] 连接设备时返回的设备句柄。
	pDevInfo	[OUT] 返回设备信息。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.1.10 锁定设备

函数原型	ULONG DEVAPI SKF_LockDev (DEVHANDLE hDev, ULONG ulTimeOut)	
功能描述	获得设备的独占使用权。	
参数	hDev	[IN] 连接设备时返回的设备句柄。
	ulTimeOut	[IN] 超时时间，单位为毫秒。如果为 0xFFFFFFFF 表示无限等待。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.1.11 解锁设备

函数原型	ULONG DEVAPI SKF_UnlockDev (DEVHANDLE hDev)	
功能描述	释放对设备的独占使用权。	
参数	hDev	[IN] 连接设备时返回的设备句柄。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.1.12 设备命令传输

函数原型	ULONG DEVAPI SKF_Transmit(DEVHANDLE hDev, BYTE* pbCommand, ULONG ulCommandLen, BYTE* pbData, ULONG* pulDataLen)		
功能描述	将命令直接发送给设备，并返回结果。		
参数	hDev	[IN]	设备句柄。
	pbCommand	[IN]	设备命令。
	ulCommandLen	[IN]	命令长度。
	pbData	[OUT]	返回结果数据。
	pulDataLen	[IN, OUT]	输入时表示结果数据缓冲区长度，输出时表示结果数据实际长度。
返回值	SAR_OK:	成功。	
	其他:	错误码。	
备注	本函数仅用于设备检测。		

3.1.2 访问控制

3.1.2.1 概述

访问控制主要完成设备认证、PIN 码管理和安全状态管理等操作。访问控制系列函数如表 17:

表 17 访问控制系列函数

函数名称	功能
SKF_ChangeDevAuthKey	修改设备认证密钥
SKF_DevAuth	设备认证
SKF_ChangePIN	修改 PIN
SKF_GetPINInfo	获得 PIN 码信息
SKF_VerifyPIN	校验 PIN
SKF_UnblockPIN	解锁 PIN
SKF_ClearSecueState	清除应用安全状态

3.1.2.2 修改设备认证密钥

函数原型	ULONG DEVAPI SKF_ChangeDevAuthKey (DEVHANDLE hDev, BYTE *pbKeyValue, ULONG ulKeyLen)		
功能描述	更改设备认证密钥。		
参数	hDev	[IN]	连接时返回的设备句柄。
	pbKeyValue	[IN]	密钥值。
	ulKeyLen	[IN]	密钥长度。
返回值	SAR_OK:	成功。	
	其他:	错误码。	

备注 权限要求：设备认证成功后才能使用。

3.1.2.3 设备认证

函数原型	ULONG DEVAPI SKF_DevAuth (DEVHANDLE hDev, BYTE *pbAuthData, ULONG ulLen)	
功能描述	设备认证是设备对应用程序的认证。认证过程参见 8.2.3。	
参数	hDev	[IN] 连接时返回的设备句柄。
	pbAuthData	[IN] 认证数据。
	ulLen	[IN] 认证数据的长度。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.2.4 修改 PIN

函数原型	ULONG DEVAPI SKF_ChangePIN (HAPPLICATION hApplication, ULONG ulPINType, LPSTR szOldPin, LPSTR szNewPin, ULONG *pulRetryCount)	
功能描述	调用该函数可以修改 Administrator PIN 和 User PIN 的值。 如果原 PIN 码错误导致验证失败，该函数会返回相应 PIN 码的剩余重试次数，当剩余次数为 0 时，表示 PIN 已经被锁死。	
参数	hApplication	[IN] 应用句柄。
	ulPINType	[IN] PIN 类型，可为 ADMIN_TYPE 或 USER_TYPE。
	szOldPin	[IN] 原 PIN 值。
	szNewPin	[IN] 新 PIN 值。
	pulRetryCount	[OUT] 出错后重试次数。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.2.5 获取 PIN 信息

函数原型	ULONG DEVAPI SKF_GetPINInfo(HAPPLICATION hApplication, ULONG ulPINType, ULONG *pulMaxRetryCount, ULONG *pulRemainRetryCount, BOOL *pbDefaultPin)	
功能描述	获取 PIN 码信息，包括最大重试次数、当前剩余重试次数，以及当前 PIN 码是否为出厂默认 PIN 码。	
参数	hApplication	[IN] 应用句柄。
	ulPINType	[IN] PIN 类型。
	pulMaxRetryCount	[OUT] 最大重试次数。
	pulRemainRetryCount	[OUT] 当前剩余重试次数，当为 0 时表示已锁死。
	pbDefaultPin	[OUT] 是否为出厂默认 PIN 码。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.2.6 校验 PIN

函数原型	ULONG DEVAPI SKF_VerifyPIN (HAPPLICATION hApplication, ULONG ulPINType, LPSTR szPIN, ULONG *pulRetryCount)
功能描述	校验 PIN 码。校验成功后，会获得相应的权限，如果 PIN 码错误，会返回 PIN 码的重试次数，当重试次数为 0 时表示 PIN 码已经锁死。
参数	hApplication [IN] 应用句柄。 ulPINType [IN] PIN 类型。 szPIN [IN] PIN 值。 pulRetryCount [OUT] 出错后返回的重试次数。
返回值	SAR_OK: 成功。 其他: 错误码。

3.1.2.7 解锁 PIN

函数原型	ULONG DEVAPI SKF_UnblockPIN (HAPPLICATION hApplication, LPSTR szAdminPIN, LPSTR szNewUserPIN, ULONG *pulRetryCount)
功能描述	当用户的 PIN 码锁死后，通过调用该函数来解锁用户 PIN 码。 解锁后，用户 PIN 码被设置成新值，用户 PIN 码的重试次数也恢复到原值。
参数	hApplication [IN] 应用句柄。 szAdminPIN [IN] 管理员 PIN 码。 szNewUserPIN [IN] 新的用户 PIN 码。 pulRetryCount [OUT] 管理员 PIN 码错误时，返回剩余重试次数。
返回值	SAR_OK: 成功。 其他: 错误码。
备注	验证完管理员 PIN 才能够解锁用户 PIN 码，如果输入的 Administrator PIN 不正确或者已经锁死，会调用失败，并返回 Administrator PIN 的重试次数。

3.1.2.8 清除应用安全状态

函数原型	ULONG DEVAPI SKF_ClearSecureState (HAPPLICATION hApplication)
功能描述	清除应用当前的安全状态。
参数	hApplication [IN] 应用句柄。
返回值	SAR_OK: 成功。 其他: 错误码。

3.1.3 应用管理

函数原型	ULONG DEVAPI SKF_CreateApplication(DEVHANDLE hDev, LPSTR szAppName, LPSTR szAdminPin, DWORD dwAdminPinRetryCount, LPSTR szUserPin, DWORD dwUserPinRetryCount, DWORD dwCreateFileRights, HAPPLICATION *phApplication)	
功能描述	创建一个应用。	
参数	hDev	[IN] 连接设备时返回的设备句柄。
	szAppName	[IN] 应用名称。
	szAdminPin	[IN] 管理员 PIN。
	dwAdminPinRetryCount	[IN] 管理员 PIN 最大重试次数。
	szUserPin	[IN] 用户 PIN。
	dwUserPinRetryCount	[IN] 用户 PIN 最大重试次数。
	dwCreateFileRights	[IN] 在该应用下创建文件和容器的权限，参见 6.4.12 权限类型。为各种权限的或值。
	phApplication	[OUT] 应用的句柄。
返回值	SAR_OK:	成功。
	其他:	错误码。
备注	权限要求：需要设备权限。	

3.1.3.1 概述

应用管理主要完成应用的创建、枚举、删除、打开、关闭等操作。应用管理系列函数如表 18 所示：

表 18 应用管理系列函数

函数名称	功能
SKF_CreateApplication	创建应用
SKF_EnumApplication	枚举应用
SKF_DeleteApplication	删除应用
SKF_OpenApplication	打开应用
SKF_CloseApplication	关闭应用

3.1.3.2 创建应用

3.1.3.3 枚举应用

函数原型	ULONG DEVAPI SKF_EnumApplication(DEVHANDLE hDev, LPSTR szAppName, ULONG *pulSize)	
功能描述	枚举设备中存在的所有应用。	
参数	hDev	[IN] 连接设备时返回的设备句柄。
	szAppName	[OUT] 返回应用名称列表，如果该参数为空，将由 pulSize 返回所

需要的内存空间大小。每个应用的名称以单个 ‘\0’ 结束，以双 ‘\0’ 表示列表的结束。

pulSize [IN, OUT] 输入时表示应用名称的缓冲区长度，输出时返回 szAppName 所占用的空间大小。

返回值 SAR_OK: 成功。
其他: 错误码。

3.1.3.4 删除应用

函数原型 `ULONG DEVAPI SKF_DeleteApplication(DEVHANDLE hDev, LPSTR szAppName)`

功能描述 删除指定的应用。

参数 hDev [IN] 连接设备时返回的设备句柄。
szAppName [IN] 应用名称。

返回值 SAR_OK: 成功。
其他: 错误码。

备注 权限要求: 需要设备权限。

3.1.3.5 打开应用

函数原型 `ULONG DEVAPI SKF_OpenApplication(DEVHANDLE hDev, LPSTR szAppName, HAPPLICATION *phApplication)`

功能描述 打开指定的应用。

参数 hDev [IN] 连接设备时返回的设备句柄。
szAppName [IN] 应用名称。
phApplication [OUT] 应用的句柄。

返回值 SAR_OK: 成功。
其他: 错误码。

3.1.3.6 关闭应用

函数原型 `ULONG DEVAPI SKF_CloseApplication(HAPPLICATION hApplication)`

功能描述 关闭应用并释放应用句柄。

参数 hApplication [IN] 应用句柄。

返回值 SAR_OK: 成功。
其他: 错误码。

备注 此函数不影响应用安全状态。

3.1.4 文件管理

3.1.4.1 概述

文件管理函数用以满足用户扩展开发的需要，包括创建文件、删除文件、枚举文件、获取文件信息、文件读写等操作。文件管理系列函数如表 19 所示：

表 19 文件管理系列函数

函数名称	功能
SKF_CreateFile	创建文件
SKF_DeleteFile	删除文件
SKF_EnumFiles	枚举文件
SKF_GetFileInfo	获取文件信息
SKF_ReadFile	读文件
SKF_WriteFile	写文件

3.1.4.2 创建文件

函数原型	ULONG DEVAPI SKF_CreateFile (HAPPLICATION hApplication, LPSTR szFileName, ULONG ulFileSize, ULONG ulReadRights, ULONG ulWriteRights)
功能描述	创建文件时要指定文件的名称，大小，以及文件的读写权限。
参数	hApplication [IN] 应用句柄。 szFileName [IN] 文件名称，长度不得大于 32 个字节。 ulFileSize [IN] 文件大小。 ulReadRights [IN] 文件读权限，参见 6.4.12 权限类型。可为各种权限的或值。 ulWriteRights [IN] 文件写权限，参见 6.4.12 权限类型。可为各种权限的或值。
返回值	SAR_OK: 成功。 其他: 错误码。
备注	创建文件需要应用指定的创建文件权限。

3.1.4.3 删除文件

函数原型	ULONG DEVAPI SKF_DeleteFile (HAPPLICATION hApplication, LPSTR szFileName)
功能描述	删除指定文件： 文件删除后，文件中写入的所有信息将丢失。 文件在设备中的占用的空间将被释放。
参数	hApplication [IN] 要删除文件所在的应用句柄。 szFileName [IN] 要删除文件的名称。
返回值	SAR_OK: 成功。 其他: 错误码。
备注	权限要求：删除一个文件应具有对该文件的创建权限。

3.1.4.4枚举文件

函数原型	ULONG DEVAPI SKF_EnumFiles (HAPPLICATION hApplication, LPSTR szFileList, ULONG *pulSize)	
功能描述	枚举一个应用下存在的所有文件。	
参数	hApplication	[IN] 应用句柄。
	szFileList	[OUT] 返回文件名称列表，该参数为空，由 pulSize 返回文件信息所需要的空间大小。每个文件名称以单个 ‘\0’ 结束，以双 ‘\0’ 表示列表的结束。
	pulSize	[IN, OUT] 输入时表示数据缓冲区的大小，输出时表示实际文件名称列表的长度。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.4.5获取文件属性

函数原型	ULONG DEVAPI SKF_GetFileInfo (HAPPLICATION hApplication, LPSTR szFileName, FILEATTRIBUTE *pFileInfo)	
功能描述	获取文件信息： 获取应用文件的属性信息，例如文件的大小、权限等。	
参数	hApplication	[IN] 文件所在应用的句柄。
	szFileName	[IN] 文件名称。
	pFileInfo	[OUT] 文件信息，指向文件属性结构的指针。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.4.6读文件

函数原型	ULONG DEVAPI SKF_ReadFile (HAPPLICATION hApplication, LPSTR szFileName, ULONG ulOffset, ULONG ulSize, BYTE * pbOutData, ULONG *pulOutLen)	
功能描述	读取文件内容。	
参数	hApplication	[IN] 应用句柄。
	szFileName	[IN] 文件名。
	ulOffset	[IN] 文件读取偏移位置。
	ulSize	[IN] 要读取的长度。
	pbOutData	[OUT] 返回数据的缓冲区。
	pulOutLen	[IN, OUT] 输入时表示给出的缓冲区大小；输出时表示实际读取返回的数据大小。
返回值	SAR_OK:	成功。
	其他:	错误码。
备注	权限要求：须具备对该文件的读权限。	

3.1.4.7 写文件

函数原型	ULONG DEVAPI SKF_WriteFile (HAPPLICATION hApplication, LPSTR szFileName, ULONG ulOffset, BYTE *pbData, ULONG ulSize)
功能描述	写数据到文件中。
参数	hApplication [IN] 应用句柄。 szFileName [IN] 文件名。 ulOffset [IN] 写入文件的偏移量。 pbData [IN] 写入数据缓冲区。 ulSize [IN] 写入数据的大小。
返回值	SAR_OK: 成功。 其他: 错误码。
备注	权限要求: 须具备该文件的写权限。

3.1.5 容器管理

3.1.5.1 概述

本规范提供的容器管理用于满足各种不同应用的管理，包括创建、删除、枚举、打开和关闭容器的操作。容器管理系列函数如表 20 所示：

表 20 容器管理系列函数

函数名称	功能
SKF_CreateContainer	创建容器
SKF_DeleteContainer	删除容器
SKF_EnumContainer	枚举容器
SKF_OpenContainer	打开容器
SKF_CloseContainer	关闭容器
SKF_GetContainerType	获得容器类型
SKF_ImportCertificate	导入数字证书
SKF_ExportCertificate	导出数字证书

3.1.5.2 创建容器

函数原型	ULONG DEVAPI SKF_CreateContainer (HAPPLICATION hApplication, LPSTR szContainerName, HCONTAINER *phContainer)
功能描述	在应用下建立指定名称的容器并返回容器句柄。
参数	hApplication [IN] 应用句柄。 szContainerName [IN] ASCII 字符串，表示所建立容器的名称，容器名称的最大长度不能超过 64 字节。 phContainer [OUT] 返回所建立容器的容器句柄。

返回值	SAR_OK: 成功。
	其他: 错误码。
备注	权限要求: 需要用户权限。

3.1.5.3删除容器

函数原型	ULONG DEVAPI SKF_DeleteContainer(HAPPLICATION hApplication, LPSTR szContainerName)
功能描述	在应用下删除指定名称的容器并释放容器相关的资源。
参数	hApplication [IN] 应用句柄。 szContainerName [IN] 指向删除容器的名称。
返回值	SAR_OK: 成功。 其他: 错误码。
备注	权限要求: 需要用户权限。

3.1.5.4打开容器

函数原型	ULONG DEVAPI SKF_OpenContainer(HAPPLICATION hApplication, LPSTR szContainerName, HCONTAINER *phContainer)
功能描述	获取容器句柄。
参数	hApplication [IN] 应用句柄。 szContainerName [IN] 容器的名称。 phContainer [OUT] 返回所打开容器的句柄。
返回值	SAR_OK: 成功。 其他: 错误码。

3.1.5.5关闭容器

函数原型	ULONG DEVAPI SKF_CloseContainer(HCONTAINER hContainer)
功能描述	关闭容器句柄，并释放容器句柄相关资源。
参数	hContainer [IN] 容器句柄。
返回值	SAR_OK: 成功。 其他: 错误码。

3.1.5.6枚举容器

3.1.5.7获得容器类型

函数原型	ULONG DEVAPI SKF_GetContainerType(HCONTAINER hContainer, ULONG *pulContainerType)
------	---

功能描述	获取容器的类型	
参数	hContainer	[IN] 容器句柄。
	pulContainerType	[OUT] 获得的容器类型。指针指向的值为 0 表示未定、尚未分配类型或者为空容器，为 1 表示为 RSA 容器，为 2 表示为 ECC 容器。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.5.8 导入数字证书

函数原型	ULONG DEVAPI SKF_ImportCertificate(HCONTAINER hContainer, BOOL bSignFlag, BYTE* pbCert, ULONG ulCertLen)	
功能描述	向容器内导入数字证书。	
参数	hContainer	[IN] 容器句柄。
	bSignFlag	[IN] TRUE 表示签名证书，FALSE 表示加密证书。
	pbCert	[IN] 指向证书内容缓冲区。
	ulCertLen	[IN] 证书长度。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.5.9 导出数字证书

函数原型	ULONG DEVAPI SKF_ExportCertificate(HCONTAINER hContainer, BOOL bSignFlag, BYTE* pbCert, ULONG *pulCertLen)	
功能描述	从容器内导出数字证书。	
参数	hContainer	[IN] 容器句柄。
	bSignFlag	[IN] TRUE 表示签名证书，FALSE 表示加密证书。
	pbCert	[OUT] 指向证书内容缓冲区，如果此参数为 NULL 时，pulCertLen 表示返回数据所需要缓冲区的长度，如果此参数不为 NULL 时，返回数字证书内容。
	pulCertLen	[IN/OUT] 输入时表示 pbCert 缓冲区的长度，输出时表示证书内容的长度。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.6 密码服务

3.1.6.1 概述

密码服务函数提供对称算法运算、非对称算法运算、密码杂凑运算、密钥管理、消息鉴别码计算等功能。密码服务系列函数如表 21 所示：

表 21 密码服务系列函数

函数名称	功能
SKF_GenRandom	生成随机数
SKF_GenExtRSAKey	生成外部 RSA 密钥对
SKF_GenRSAKeyPair	生成 RSA 签名密钥对
SKF_ImportRSAKeyPair	导入 RSA 加密密钥对
SKF_RSASignData	RSA 签名
SKF_RSAVerify	RSA 验签
SKF_RSAExportSessionKey	RSA 生成并导出会话密钥
SKF_ExtRSAPubKeyOperation	RSA 外来公钥运算
SKF_ExtRSAPriKeyOperation	RSA 外来私钥运算
SKF_GenECCKeyPair	生成 ECC 签名密钥对
SKF_ImportECCKeyPair	导入 ECC 加密密钥对
SKF_ECCSignData	ECC 签名
SKF_ECCVerify	ECC 验签
SKF_ECCExportSessionKey	ECC 生成并导出会话密钥
SKF_ExtECCEncrypt	ECC 外来公钥加密
SKF_ExtECCDecrypt	ECC 外来私钥解密
SKF_ExtECCSign	ECC 外来私钥签名
SKF_ExtECCVerify	ECC 外来公钥验签
SKF_GenerateAgreementDataWithECC	ECC 生成密钥协商参数并输出
SKF_GenerateKeyWithECC	ECC 计算会话密钥
SKF_GenerateAgreementDataAndKeyWithECC	ECC 产生协商数据并计算会话密钥
SKF_ExportPublicKey	导出公钥
SKF_ImportSessionKey	导入会话密钥
SKF_SetSymmKey	明文导入会话密钥
SKF_EncryptInit	加密初始化
SKF_Encrypt	单组数据加密
SKF_EncryptUpdate	多组数据加密
SKF_EncryptFinal	结束加密
SKF_DecryptInit	解密初始化
SKF_Decrypt	单组数据解密
SKF_DecryptUpdate	多组数据解密
SKF_DecryptFinal	结束解密
SKF_DigestInit	密码杂凑初始化
SKF_Digest	单组数据密码杂凑
SKF_DigestUpdate	多组数据密码杂凑
SKF_DigestFinal	结束密码杂凑
SKF_MacInit	消息鉴别码运算初始化
SKF_Mac	单组数据消息鉴别码运算
SKF_MacUpdate	多组数据消息鉴别码运算
SKF_MacFinal	结束消息鉴别码运算

SKF_CloseHandle

关闭密码对象句柄

3.1.6.2生成随机数

函数原型 ULONG DEVAPI SKF_GenRandom (DEVHANDLE hDev, BYTE *pbRandom, ULONG ulRandomLen)

功能描述 产生指定长度的随机数。

参数

hDev	[IN] 设备句柄。
pbRandom	[OUT]返回的随机数。
ulRandomLen	[IN] 随机数长度。

返回值

SAR_OK:	成功。
其他:	错误码。

3.1.6.3生成外部 RSA 密钥对

函数原型 ULONG DEVAPI SKF_GenExtRSAKey (DEVHANDLE hDev, ULONG ulBitsLen, RSAPRIVATEKEYBLOB *pBlob)

功能描述 由设备生成 RSA 密钥对并明文输出。

参数

hDev	[IN]设备句柄。
ulBitsLen	[IN] 密钥模长。
pBlob	[OUT] 返回的私钥数据结构。

返回值

SAR_OK:	成功。
其他:	错误码。

备注: 生成的私钥只用于输出，接口内不做保留和计算。

3.1.6.4生成 RSA 签名密钥对

函数原型 ULONG DEVAPI SKF_GenRSAKeyPair (HCONTAINER hContainer, ULONG ulBitsLen, RSAPUBLICKEYBLOB *pBlob)

功能描述 生成 RSA 签名密钥对并输出签名公钥。

参数

hContainer	[IN] 容器句柄。
ulBitsLen	[IN] 密钥模长。
pBlob	[OUT] 返回的 RSA 公钥数据结构。

返回值

SAR_OK:	成功。
其他:	错误码。

备注 权限要求：需要用户权限。

3.1.6.5导入 RSA 加密密钥对

函数原型 ULONG DEVAPI SKF_ImportRSAKeyPair (HCONTAINER hContainer, ULONG ulSymAlgId,

	BYTE *pbWrappedKey, ULONG ulWrappedKeyLen, BYTE *pbEncryptedData, ULONG ulEncryptedDataLen)	
功能描述	导入 RSA 加密公私钥对。	
参数	hContainer	[IN] 容器句柄。
	ulSymAlgId	[IN] 对称算法密钥标识。
	pbWrappedKey	[IN] 使用该容器内签名公钥保护的对称算法密钥。
	ulWrappedKeyLen	[IN] 保护的对称算法密钥长度。
	pbEncryptedData	[IN] 对称算法密钥保护的 RSA 加密私钥。私钥的格式遵循 PKCS #1 v2.1: RSA Cryptography Standard 中的私钥格式定义。
	ulEncryptedDataLen	[IN] 对称算法密钥保护的 RSA 加密公私钥对长度。
返回值	SAR_OK:	成功。
	其他:	错误码。
备注	权限要求: 需要用户权限。	

3.1.6.6 RSA 签名

函数原型	ULONG DEVAPI SKF_RSASignData(HCONTAINER hContainer, BYTE *pbData, ULONG ulDataLen, BYTE *pbSignature, ULONG *pulSignLen)	
功能描述	使用 hContainer 指定容器的签名私钥, 对指定数据 pbData 进行数字签名。签名后的结果存放到 pbSignature 缓冲区, 设置 pulSignLen 为签名的长度。	
参数	hContainer	[IN] 用来签名的私钥所在容器句柄。
	pbData	[IN] 被签名的数据。
	ulDataLen	[IN] 签名数据长度, 应不大于 RSA 密钥模长-11。
	pbSignature	[OUT] 存放签名结果的缓冲区指针, 如果值为 NULL, 用于取得签名结果长度。
	pulSignLen	[IN, OUT] 输入时表示签名结果缓冲区大小, 输出时表示签名结果长度。
返回值	SAR_OK:	成功。
	其他:	错误码。
备注	权限要求: 需要用户权限。	

3.1.6.7 RSA 验签

函数原型	ULONG DEVAPI SKF_RSASignData(HCONTAINER hContainer, BYTE *pbData, ULONG ulDataLen, BYTE *pbSignature, ULONG *pulSignLen)	
功能描述	验证 RSA 签名。用 pRSAPubKeyBlob 内的公钥值对待验签数据进行验签。	
参数	hDev	[IN] 设备句柄。
	pRSAPubKeyBlob	[IN] RSA 公钥数据结构。
	pbData	[IN] 待验证签名的数据。
	ulDataLen	[IN] 数据长度, 应不大于公钥模长-11。
	pbSignature	[IN] 待验证的签名值。
	ulSignLen	[IN] 签名值长度, 必须为公钥模长。
返回值	SAR_OK:	成功。

其他： 错误码。

3.1.6.8 RSA 生成并导出会话密钥

函数原型	ULONG DEVAPI SKF_RSASessionKey (HCONTAINER hContainer, ULONG ulAlgId, RSAPUBLICKEYBLOB *pPubKey, BYTE *pbData, ULONG *pulDataLen, HANDLE *phSessionKey)		
功能描述	生成会话密钥并用外部 RSA 公钥加密输出。		
参数	hContainer	[IN] 容器句柄。	
	ulAlgId	[IN] 会话密钥算法标识。	
	pPubKey	[IN] 加密会话密钥的 RSA 公钥数据结构。	
	pbData	[OUT] 导出的加密会话密钥密文，按照 PKCS#1v1.5 要求封装。	
	pulDataLen	[IN, OUT] 输入时表示会话密钥密文数据缓冲区长度，输出时表示会话密钥密文的实际长度。	
返回值	phSessionKey	[OUT] 导出的密钥句柄。	
	SAR_OK:	成功。	
	其他:	错误码。	

3.1.6.9 RSA 外来公钥运算

函数原型	ULONG DEVAPI SKF_ExtRSAPubKeyOperation (DEVHANDLE hDev, RSAPUBLICKEYBLOB* pRSAPubKeyBlob, BYTE* pbInput, ULONG ulInputLen, BYTE* pbOutput, ULONG* pulOutputLen)		
功能描述	使用外部传入的 RSA 公钥对输入数据做公钥运算并输出结果。		
参数	hDev	[IN] 设备句柄。	
	pRSAPubKeyBlob	[IN] RSA 公钥数据结构。	
	pbInput	[IN] 指向待运算的原始数据缓冲区。	
	ulInputLen	[IN] 待运算原始数据的长度，必须为公钥模长。	
	pbOutput	[OUT] 指向 RSA 公钥运算结果缓冲区，如果该参数为 NULL，则由 pulOutputLen 返回运算结果的实际长度。	
返回值	pulOutputLen	[IN, OUT] 输入时表示 pbOutput 缓冲区的长度，输出时表示 RSA 公钥运算结果的实际长度。	
	SAR_OK:	成功。	
	其他:	错误码。	

3.1.6.10 RSA 外来私钥运算

函数原型	ULONG DEVAPI SKF_ExtRSAPriKeyOperation (DEVHANDLE hDev, RSAPRIVATEKEYBLOB* pRSAPriKeyBlob, BYTE* pbInput, ULONG ulInputLen, BYTE* pbOutput, ULONG* pulOutputLen)		
功能描述	直接使用外部传入的 RSA 私钥对输入数据做私钥运算并输出结果。		

参数	hDev	[IN] 设备句柄。
	pRSAPriKeyBlob	[IN] RSA 私钥数据结构。
	pbInput	[IN] 指向待运算数据缓冲区。
	ulInputLen	[IN] 待运算数据的长度，必须为公钥模长。
	pbOutput	[OUT] RSA 私钥运算结果，如果该参数为 NULL，则由 pulOutputLen 返回运算结果的实际长度。
	pulOutputLen	[IN, OUT] 输入时表示 pbOutput 缓冲区的长度，输出时表示 RSA 私钥运算结果的实际长度。
返回值	SAR_OK:	成功。
	其他:	错误码。
备注	本函数仅用于测试和调试，不建议用于实际的密码服务。	

3.1.6.11 生成 ECC 签名密钥对

函数原型	ULONG DEVAPI SKF_GenECCKeyPair (HCONTAINER hContainer, ULONG ulAlgId, ECCPUBLICKEYBLOB *pBlob)	
功能描述	生成 ECC 签名密钥对并输出签名公钥。	
参数	hContainer	[IN] 密钥容器句柄。
	ulAlgId	[IN] 算法标识，只支持 SGD_SM2_1 算法。
	pBlob	[OUT] 返回 ECC 公钥数据结构。
返回值	SAR_OK:	成功。
	其他:	错误码。
备注	权限要求：需要用户权限。	

3.1.6.12 导入 ECC 加密密钥对

函数原型	ULONG DEVAPI SKF_ImportECCKeyPair (HCONTAINER hContainer, PENVELOPEDKEYBLOB pEnvelopedKeyBlob)	
功能描述	导入 ECC 公私钥对。	
参数	hContainer	[IN] 密钥容器句柄。
	pEnvelopedKeyBlob	[IN] 受保护的加密密钥对。
返回值	SAR_OK:	成功。
	其他:	错误码。
备注	权限要求：需要用户权限。	

3.1.6.13 ECC 签名

函数原型	ULONG DEVAPI SKF_ECCECCSignData (HCONTAINER hContainer, BYTE *pbData, ULONG ulDataLen, PECCSIGNATUREBLOB pSignature)	
功能描述	ECC 数字签名。采用 ECC 算法和指定私钥 hKey，对指定数据 pbData 进行数字签名。签名后的结果存放到 pSignature 中。	
参数	hContainer	[IN] 密钥容器句柄。

	pbData	[IN] 待签名的数据。
	ulDataLen	[IN] 待签名数据长度，必须小于密钥模长。
	pSignature	[OUT] 签名值。
返回值	SAR_OK:	成功。
	其他:	错误码。
备注	权限要求：需要用户权限。 输入数据为待签数据的杂凑值。当使用 SM2 算法时，该输入数据为待签数据经过 SM2 签名预处理的结果，预处理过程遵循《公钥密码基础设施应用技术体系 SM2 算法密码使用规范》。	

3.1.6.14 ECC 验签

函数原型	ULONG DEVAPI SKF_ECCVerify (DEVHANDLE hDev , ECCPUBLICKEYBLOB* pECCPubKeyBlob, BYTE *pbData, ULONG ulDataLen, PECCSIGNATUREBLOB pSignature)	
功能描述	用 ECC 公钥对数据进行验签。	
参数	hDev	[IN] 设备句柄。
	pECCPubKeyBlob	[IN] ECC 公钥数据结构。
	pbData	[IN] 待验证签名的数据。
	ulDataLen	[IN] 数据长度。
	pSignature	[IN] 待验证签名值。
返回值	SAR_OK:	成功。
	其他:	错误码。
备注	输入数据为待签数据的杂凑值。当使用 SM2 算法时，该输入数据为待签数据经过 SM2 签名预处理的结果，预处理过程遵循《公钥密码基础设施应用技术体系 SM2 算法密码使用规范》。	

3.1.6.15 ECC 生成并导出会话密钥

函数原型	ULONG DEVAPI SKF_ECCEXportSessionKey (HCONTAINER hContainer, ULONG ulAlgId, ECCPUBLICKEYBLOB *pPubKey, PECCCIPHERBLOB pData, HANDLE *phSessionKey)	
功能描述	生成会话密钥并用外部公钥加密导出。	
参数	hContainer	[IN] 容器句柄。
	ulAlgId	[IN] 会话密钥算法标识。
	pPubKey	[IN] 外部输入的公钥结构。
	pData	[OUT] 会话密钥密文。
	phSessionKey	[OUT] 会话密钥句柄。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.6.16 ECC 外来公钥加密

函数原型	ULONG DEVAPI SKF_ExtECCEncrypt (DEVHANDLE hDev, ECCPUBLICKEYBLOB* pECCPubKeyBlob, BYTE* pbPlainText, ULONG ulPlainTextLen, PECCIPHERBLOB pCipherText)		
功能描述	使用外部传入的 ECC 公钥对输入数据做加密运算并输出结果。		
参数	hDev	[IN]	设备句柄。
	pECCPubKeyBlob	[IN]	ECC 公钥数据结构。
	pbPlainText	[IN]	待加密的明文数据。
	ulPlainTextLen	[IN]	待加密明文数据的长度。
	pCipherText	[OUT]	密文数据。
返回值	SAR_OK:	成功。	
	其他:	错误码。	

3.1.6.17 ECC 外来私钥解密

函数原型	ULONG DEVAPI SKF_ExtECCDecrypt (DEVHANDLE hDev, ECCPRIVATEKEYBLOB* pECCPriKeyBlob, PECCIPHERBLOB pCipherText, BYTE* pbPlainText, ULONG* pulPlainTextLen)		
功能描述	使用外部传入的 ECC 私钥对输入数据做解密运算并输出结果。		
参数	hDev	[IN]	设备句柄。
	pECCPriKeyBlob	[IN]	ECC 私钥数据结构。
	pCipherText	[IN]	待解密的密文数据。
	pbPlainText	[OUT]	返回明文数据，如果该参数为 NULL，则由 pulPlainTextLen 返回明文数据的实际长度。
	pulPlainTextLen	[IN, OUT]	输入时表示 pbPlainText 缓冲区的长度，输出时表示明文数据的实际长度。
返回值	SAR_OK:	成功。	
	其他:	错误码。	
备注	本函数仅用于测试和调试，不建议用于实际的密码服务。		

3.1.6.18 ECC 外来私钥签名

函数原型	ULONG DEVAPI SKF_ExtECCSign (DEVHANDLE hDev, ECCPRIVATEKEYBLOB* pECCPriKeyBlob, BYTE* pbData, ULONG ulDataLen, PECCSIGNATUREBLOB pSignature)		
功能描述	使用外部传入的 ECC 私钥对输入数据做签名运算并输出结果。		
参数	hDev	[IN]	设备句柄。
	pECCPriKeyBlob	[IN]	ECC 私钥数据结构。
	pbData	[IN]	待签名数据。
	ulDataLen	[IN]	待签名数据的长度。
	pSignature	[OUT]	签名值。

返回值	SAR_OK: 成功。 其他: 错误码。
备注:	输入数据为待签数据的杂凑值。当使用 SM2 算法时, 该输入数据为待签数据经过 SM2 签名预处理的结果, 预处理过程遵循《公钥密码基础设施应用技术体系 SM2 算法密码使用规范》。
备注	本函数仅用于测试和调试, 不建议用于实际的密码服务。

3.1.6.19 ECC 外来公钥验签

函数原型	ULONG DEVAPI SKF_ExtECCVerify (DEVHANDLE hDev, ECCPUBLICKEYBLOB* pECCPubKeyBlob, BYTE* pbData, ULONG ulDataLen, PECCSIGNATUREBLOB pSignature)		
功能描述	外部使用传入的 ECC 公钥做签名验证。		
参数	hDev	[IN]	设备句柄。
	pECCPubKeyBlob	[IN]	ECC 公钥数据结构。
	pbData	[IN]	待验证数据。
	ulDataLen	[IN]	待验证数据的长度。
	pSignature	[IN]	签名值。
返回值	SAR_OK:	成功。	
	其他:	错误码。	
备注:	输入数据为待签数据的杂凑值。当使用 SM2 算法时, 该输入数据为待签数据经过 SM2 签名预处理的结果, 预处理过程遵循《公钥密码基础设施应用技术体系 SM2 算法密码使用规范》。		

3.1.6.20 ECC 生成密钥协商参数并输出

函数原型	ULONG DEVAPI SKF_GenerateAgreementDataWithECC (HCONTAINER hContainer, ULONG ulAlgId, ECCPUBLICKEYBLOB* pTempECCPubKeyBlob, BYTE* pbID, ULONG ulIDLen, HANDLE *phAgreementHandle)		
功能描述	使用 ECC 密钥协商算法, 为计算会话密钥而产生协商参数, 返回临时 ECC 密钥对的公钥及协商句柄。		
参数	hContainer	[IN]	容器句柄。
	ulAlgId	[IN]	会话密钥算法标识。
	pTempECCPubKeyBlob	[OUT]	发起方临时 ECC 公钥。
	pbID	[IN]	发起方的 ID。
	ulIDLen	[IN]	发起方 ID 的长度, 不大于 32。
	phAgreementHandle	[OUT]	返回的密钥协商句柄。
返回值	SAR_OK:	成功。	
	其他:	错误码。	
备注	为协商会话密钥, 协商的发起方应首先调用本函数。		

3.1.6.21 ECC 产生协商数据并计算会话密钥

函数原型:	ULONG DEVAPI SKF_GenerateAgreementDataAndKeyWithECC(HANDLE hContainer, ULONG ulAlgId, ECCPUBLICKEYBLOB* pSponsorECCPubKeyBlob, ECCPUBLICKEYBLOB* pSponsorTempECCPubKeyBlob, ECCPUBLICKEYBLOB* pTempECCPubKeyBlob, BYTE* pbID, ULONG ulIDLen, BYTE *pbSponsorID, ULONG ulSponsorIDLen, HANDLE *phKeyHandle)		
功能描述:	使用 ECC 密钥协商算法, 产生协商参数并计算会话密钥, 输出临时 ECC 密钥对公钥, 并返回产生的密钥句柄。		
参数:	hContainer	[IN]	容器句柄。
	ulAlgId	[IN]	会话密钥算法标识。
	pSponsorECCPubKeyBlob	[IN]	发起方的 ECC 公钥。
	pSponsorTempECCPubKeyBlob	[IN]	发起方的临时 ECC 公钥。
	pTempECCPubKeyBlob	[OUT]	响应方的临时 ECC 公钥。
	pbID	[IN]	响应方的 ID。
	ulIDLen	[IN]	响应方 ID 的长度, 不大于 32。
	pbSponsorID	[IN]	发起方的 ID。
	ulSponsorIDLen	[IN]	发起方 ID 的长度, 不大于 32。
	phKeyHandle	[OUT]	返回的对称算法密钥句柄。
返回值	SAR_OK:	成功。	
	其他:	错误码。	
备注:	本函数由响应方调用。		

3.1.6.22 ECC 计算会话密钥

函数原型:	ULONG DEVAPI SKF_GenerateKeyWithECC (HANDLE hAgreementHandle, ECCPUBLICKEYBLOB* pECCPubKeyBlob, ECCPUBLICKEYBLOB* pTempECCPubKeyBlob, BYTE* pbID, ULONG ulIDLen, HANDLE *phKeyHandle)		
功能描述:	使用 ECC 密钥协商算法, 使用自身协商句柄和响应方的协商参数计算会话密钥, 同时返回会话密钥句柄。		
参数:	hAgreementHandle	[IN]	密钥协商句柄。
	pECCPubKeyBlob	[IN]	外部输入的响应方 ECC 公钥。
	pTempECCPubKeyBlob	[IN]	外部输入的响应方临时 ECC 公钥。
	pbID	[IN]	响应方的 ID。
	ulIDLen	[IN]	响应方 ID 的长度, 不大于 32。
	phKeyHandle	[OUT]	返回的密钥句柄。
返回值	SAR_OK:	成功。	
	其他:	错误码。	
备注:	协商的发起方获得响应方的协商参数后调用本函数, 计算会话密钥。计算过程遵循《公钥密码基础设施应用技术体系 SM2 算法密码使用规范》。		

3.1.6.23 导出公钥

函数原型	ULONG DEVAPI SKF_ExportPublicKey (HCONTAINER hContainer, BOOL bSignFlag, BYTE* pbBlob, ULONG* pulBlobLen)		
功能描述	导出容器中的签名公钥或者加密公钥。		
参数	hContainer	[IN]	密钥容器句柄。
	bSignFlag	[IN]	TRUE 表示导出签名公钥, FALSE 表示导出加密公钥。
	pbBlob	[OUT]	指向 RSA 公钥结构 (RSAPUBLICKEYBLOB) 或者 ECC 公钥结构 (ECCPUBLICKEYBLOB), 如果此参数为 NULL 时, 由 pulBlobLen 返回 pbBlob 的长度。
	pulBlobLen	[IN, OUT]	输入时表示 pbBlob 缓冲区的长度, 输出时表示导出公钥结构的大小。
返回值	SAR_OK:	成功。	
	其他:	错误码。	

3.1.6.24 导入会话密钥

函数原型	ULONG DEVAPI SKF_ImportSessionKey (HCONTAINER hContainer, ULONG ulAlgId, BYTE *pbWrapedData, ULONG ulWrapedLen, HANDLE *phKey)		
功能描述	导入会话密钥密文，使用容器中的加密私钥解密得到会话密钥。		
参数	hContainer	[IN]	容器句柄。
	ulAlgId	[IN]	会话密钥算法标识。
	pbWrapedData	[IN]	要导入的会话密钥密文。当容器为 ECC 类型时，此参数为 ECCIPHERBLOB 密文数据，当容器为 RSA 类型时，此参数为 RSA 公钥加密后的数据。
	ulWrapedLen	[IN]	会话密钥密文长度。
	phKey	[OUT]	返回会话密钥句柄。
返回值	SAR_OK:	成功。	
	其他:	错误码。	
备注	权限要求：需要用户权限。		

3.1.6.25 明文导入会话密钥

函数原型	ULONG DEVAPI SKF_SetSymmKey (DEVHANDLE hDev, BYTE* pbKey, ULONG ulAlgID, HANDLE* phKey)		
功能描述	设置明文对称密钥, 返回密钥句柄。		
参数	hDev	[IN]	设备句柄。
	pbKey	[IN]	指向会话密钥值的缓冲区。
	ulAlgID	[IN]	会话密钥算法标识。
	phKey	[OUT]	返回会话密钥句柄。
返回值	SAR_OK:	成功。	
	其他:	错误码。	

备注 本函数仅用于测试和调试，不建议用于实际的密码服务。

3.1.6.26 加密初始化

函数原型 `ULONG DEVAPI SKF_EncryptInit (HANDLE hKey, BLOCKCIPHERPARAM EncryptParam)`

功能描述 数据加密初始化。设置数据加密的算法相关参数。

参数 `hKey` [IN] 加密密钥句柄。
`EncryptParam` [IN] 分组密码算法相关参数：初始向量、初始向量长度、填充方法、反馈值的位长度。

返回值 `SAR_OK`: 成功。
 其他: 错误码。

3.1.6.27 单组数据加密

函数原型 `ULONG DEVAPI SKF_Encrypt (HANDLE hKey, BYTE * pbData, ULONG ulDataLen, BYTE *pbEncryptedData, ULONG *pulEncryptedLen)`

功能描述 单一分组数据的加密操作。用指定加密密钥对指定数据进行加密，被加密的数据只包含一个分组，加密后的密文保存到指定的缓冲区中。`SKF_Encrypt` 只对单个分组数据进行加密，在调用 `SKF_Encrypt` 之前，必须调用 `SKF_EncryptInit` 初始化加密操作。`SKF_Encrypt` 等价于先调用 `SKF_EncryptUpdate` 再调用 `SKF_EncryptFinal`。

参数 `hKey` [IN] 加密密钥句柄。
`pbData` [IN] 待加密数据。
`ulDataLen` [IN] 待加密数据长度。
`pbEncryptedData` [OUT] 加密后的数据缓冲区指针，可以为 `NULL`，用于获得加密后数据长度。
`pulEncryptedLen` [IN, OUT] 输入时表示结果数据缓冲区长度，输出时表示结果数据实际长度。

返回值 `SAR_OK`: 成功。
 其他: 错误码。

3.1.6.28 多组数据加密

函数原型 `ULONG DEVAPI SKF_EncryptUpdate (HANDLE hKey, BYTE * pbData, ULONG ulDataLen, BYTE *pbEncryptedData, ULONG *pulEncryptedLen)`

功能描述 多个分组数据的加密操作。用指定加密密钥对指定数据进行加密，被加密的数据包含多个分组，加密后的密文保存到指定的缓冲区中。`SKF_EncryptUpdate` 对多个分组数据进行加密，在调用 `SKF_EncryptUpdate` 之前，必须调用 `SKF_EncryptInit` 初始化加密操作；在调用 `SKF_EncryptUpdate` 之后，必须调用 `SKF_EncryptFinal` 结束加密操作。

参数 `hKey` [IN] 加密密钥句柄。
`pbData` [IN] 待加密数据。
`ulDataLen` [IN] 待加密数据长度。

	pbEncryptedData	[OUT] 加密后的数据缓冲区指针。
	pulEncryptedLen	[OUT] 返回加密后的数据长度。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.6.29 结束加密

函数原型	ULONG DEVAPI SKF_EncryptFinal (HANDLE hKey, BYTE *pbEncryptedData, ULONG *ulEncryptedDataLen)	
功能描述	结束多个分组数据的加密，返回剩余加密结果。先调用 SKF_EncryptInit 初始化加密操作，再调用 SKF_EncryptUpdate 对多个分组数据进行加密，最后调用 SKF_EncryptFinal 结束多个分组数据的加密。	
参数	hKey	[IN] 加密密钥句柄。
	pbEncryptedData	[OUT] 加密结果的缓冲区。
	ulEncryptedDataLen	[OUT] 加密结果的长度。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.6.30 解密初始化

函数原型	ULONG DEVAPI SKF_DecryptInit (HANDLE hKey, BLOCKCIPHERPARAM DecryptParam)	
功能描述	数据解密初始化，设置解密密钥相关参数。调用 SKF_DecryptInit 之后，可以调用 SKF_Decrypt 对单个分组数据进行解密，也可以多次调用 SKF_DecryptUpdate 之后再调用 SKF_DecryptFinal 完成对多个分组数据的解密。	
参数	hKey	[IN] 解密密钥句柄。
	DecryptParam	[IN] 分组密码算法相关参数：初始向量、初始向量长度、填充方法、反馈值的位长度。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.6.31 单组数据解密

函数原型	ULONG DEVAPI SKF_Decrypt(HANDLE hKey, BYTE * pbEncryptedData, ULONG ulEncryptedLen, BYTE * pbData, ULONG * pulDataLen)	
功能描述	单个分组数据的解密操作。用指定解密密钥对指定数据进行解密，被解密的数据只包含一个分组，解密后的明文保存到指定的缓冲区中。SKF_Decrypt 只对单个分组数据进行解密，在调用 SKF_Decrypt 之前，必须调用 SKF_DecryptInit 初始化解密操作。SKF_Decrypt 等价于先调用 SKF_DecryptUpdate 再调用 SKF_DecryptFinal。	
参数	hKey	[IN] 解密密钥句柄。
	pbEncryptedData	[IN] 待解密数据。
	ulEncryptedLen	[IN] 待解密数据长度。

	pbData	[OUT] 指向解密后的数据缓冲区指针，当为 NULL 时可获得解密后的数据长度。
	pulDataLen	[IN, OUT] 输入时表示结果数据缓冲区长度，输出时表示结果数据实际长度。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.6.32 多组数据解密

函数原型	ULONG DEVAPI SKF_DecryptUpdate(HANDLE hKey, BYTE * pbEncryptedData, ULONG ulEncryptedLen, BYTE * pbData, ULONG * pulDataLen)	
功能描述	多个分组数据的解密操作。用指定解密密钥对指定数据进行解密，被解密的数据包含多个分组，解密后的明文保存到指定的缓冲区中。SKF_DecryptUpdate 对多个分组数据进行解密，在调用 SKF_DecryptUpdate 之前，必须调用 SKF_DecryptInit 初始化解密操作；在调用 SKF_DecryptUpdate 之后，必须调用 SKF_DecryptFinal 结束解密操作。	
参数	hKey	[IN] 解密密钥句柄。
	pbEncryptedData	[IN] 待解密数据。
	ulEncryptedLen	[IN] 待解密数据长度。
	pbData	[OUT] 指向解密后的数据缓冲区指针。
	pulDataLen	[IN, OUT] 输入时表示结果数据缓冲区长度，输出时表示结果数据实际长度。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.6.33 结束解密

函数原型	ULONG DEVAPI SKF_DecryptFinal (HANDLE hKey, BYTE *pbDecryptedData, ULONG *pulDecryptedDataLen)	
功能描述	结束多个分组数据的解密。先调用 SKF_DecryptInit 初始化解密操作，再调用 SKF_DecryptUpdate 对多个分组数据进行解密，最后调用 SKF_DecryptFinal 结束多个分组数据的解密。	
参数	hKey	[IN] 解密密钥句柄。
	pbDecryptedData	[OUT] 指向解密结果的缓冲区，如果此参数为 NULL 时，由 pulDecryptedDataLen 返回解密结果的长度。
	pulDecryptedDataLen	[IN, OUT] 输入时表示 pbDecryptedData 缓冲区的长度，输出时表示解密结果的长度。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.6.34 密码杂凑初始化

函数原型	ULONG DEVAPI SKF_DigestInit(DEVHANDLE hDev, ULONG ulAlgID,
------	--

	ECCPUBLICKEYBLOB *pPubKey, unsigned char *pucID, ULONG ulIDLen, HANDLE *phHash)	
功能描述	初始化密码杂凑计算操作，指定计算密码杂凑的算法。	
参数	hDev	[IN] 连接设备时返回的设备句柄。
	ulAlgID	[IN] 密码杂凑算法标识。
	pPubKey	[IN] 签名者公钥。当 ulAlgID 为 SGD_SM3 时有效。
	pucID	[IN] 签名者的 ID 值，当 ulAlgID 为 SGD_SM3 时有效。
	ulIDLen	[IN] 签名者 ID 的长度，当 ulAlgID 为 SGD_SM3 时有效。
	phHash	[OUT] 密码杂凑对象句柄。
返回值	SAR_OK:	成功。
	其他:	错误码。
备注	当 ulAlgID 为 SGD_SM3 且 ulIDLen 不为 0 的情况下 pPubKey、pucID 有效，执行 SM2 算法签名预处理 1 操作。计算过程遵循《公钥密码基础设施应用技术体系 SM2 算法密码使用规范》。	

3.1.6.35 单组数据密码杂凑

函数原型	ULONG DEVAPI SKF_Digest (HANDLE hHash, BYTE *pbData, ULONG ulDataLen, BYTE *pbHashData, ULONG *pulHashLen)	
功能描述	对单分组的消息进行密码杂凑计算。调用 SKF_Digest 之前，必须调用 SKF_DigestInit 初始化密码杂凑计算操作。SKF_Digest 等价于多次调用 SKF_DigestUpdate 之后再调用 SKF_DigestFinal。	
参数	hHash	[IN] 密码杂凑对象句柄。
	pbData	[IN] 指向消息数据的缓冲区。
	ulDataLen	[IN] 消息数据的长度。
	pbHashData	[OUT] 密码杂凑数据缓冲区指针，当此参数为 NULL 时，由 pulHashLen 返回密码杂凑结果的长度。
	pulHashLen	[IN, OUT] 输入时表示结果数据缓冲区长度，输出时表示结果数据实际长度。
返回值	SAR_OK:	成功。
	其他:	错误码。

3.1.6.36 多组数据密码杂凑

函数原型	ULONG DEVAPI SKF_DigestUpdate (HANDLE hHash, BYTE *pbData, ULONG ulDataLen)	
功能描述	对多个分组的消息进行密码杂凑计算。调用 SKF_DigestUpdate 之前，必须调用 SKF_DigestInit 初始化密码杂凑计算操作；调用 SKF_DigestUpdate 之后，必须调用 SKF_DigestFinal 结束密码杂凑计算操作。	
参数	hHash	[IN] 密码杂凑对象句柄。
	pbData	[IN] 指向消息数据的缓冲区。
	ulDataLen	[IN] 消息数据的长度。

返回值 SAR_OK: 成功。
其他: 错误码。

3.1.6.37 结束密码杂凑

函数原型 ULONG DEVAPI SKF_DigestFinal (HANDLE hHash, BYTE *pHashData, ULONG *pulHashLen)

功能描述 结束多个分组消息的密码杂凑计算操作，将密码杂凑结果保存到指定的缓冲区。

参数 hHash [IN] 密码杂凑对象句柄。
pHashData [OUT] 返回的密码杂凑结果缓冲区指针，如果此参数 NULL 时，由 pulHashLen 返回杂凑结果的长度。
pulHashLen [IN, OUT] 输入时表示杂凑结果缓冲区的长度，输出时表示密码杂凑结果的长度。

返回值 SAR_OK: 成功。
其他: 错误码。

备注 SKF_DigestFinal 必须用于 SKF_DigestUpdate 之后。

3.1.6.38 消息鉴别码运算初始化

函数原型 ULONG DEVAPI SKF_MacInit (HANDLE hKey, BLOCKCIPHERPARAM* pMacParam, HANDLE *phMac)

功能描述 初始化消息鉴别码计算操作，设置计算消息鉴别码的所需参数，并返回消息鉴别码句柄。

参数 hKey [IN] 计算消息鉴别码的密钥句柄。
pMacParam [IN] 消息认证计算相关参数，包括初始向量、初始向量长度、填充方法等。
phMac [OUT] 消息鉴别码对象句柄。

返回值 SAR_OK: 成功。
其他: 错误码。

备注 消息鉴别码计算采用分组加密算法的 CBC 模式，将加密结果的最后一块作为计算结果。待计算数据的长度必须是分组加密算法块长的倍数，接口内部不作数据填充。

3.1.6.39 单组数据消息鉴别码运算

函数原型 ULONG DEVAPI SKF_Mac(HANDLE hMac, BYTE* pbData, ULONG ulDataLen, BYTE *pbMacData, ULONG *pulMacLen)

功能描述 SKF_Mac 计算单一分组数据的消息鉴别码。

参数 hMac [IN] 消息鉴别码句柄。
pbData [IN] 指向待计算数据的缓冲区。
ulDataLen [IN] 待计算数据的长度。
pbMacData [OUT] 指向计算后的 Mac 结果，如果此参数为 NULL 时，由 pulMacLen 返回计算后 Mac 结果的长度。
pulMacLen [IN, OUT] 输入时表示 pbMacData 缓冲区的长度，输出时表示 Mac 结果

		的长度。
返回值	SAR_OK:	成功。
	其他:	错误码。
备注	调用 SKF_Mac 之前，必须调用 SKF_MacInit 初始化消息鉴别码计算操作。SKF_Mac 等价于多次调用 SKF_MacUpdate 之后再调用 SKF_MacFinal。	

3.1.6.40 多组数据消息鉴别码运算

函数原型	ULONG DEVAPI SKF_MacUpdate(HANDLE hMac, BYTE * pbData, ULONG ulDataLen)	
功能描述	计算多个分组数据的消息鉴别码。	
参数	hMac	[IN] 消息鉴别码句柄。
	pbData	[IN] 指向待计算数据的缓冲区。
	plDataLen	[IN] 待计算数据的长度。
返回值	SAR_OK:	成功。
	其他:	错误码。
备注	调用 SKF_MacUpdate 之前，必须调用 SKF_MacInit 初始化消息鉴别码计算操作；调用 SKF_MacUpdate 之后，必须调用 SKF_MacFinal 结束多个分组数据的消息鉴别码计算操作。	

3.1.6.41 结束消息鉴别码运算

函数原型	ULONG DEVAPI SKF_MacFinal (HANDLE hMac, BYTE *pbMacData, ULONG *pulMacDataLen)	
功能描述	结束多个分组数据的消息鉴别码计算操作。	
参数	hMac	[IN] 消息鉴别码句柄。
	pbMacData	[OUT] 指向消息鉴别码的缓冲区，当此参数为 NULL 时，由 pulMacDataLen 返回消息鉴别码返回的长度。
	pulMacDataLen	[OUT] 调用时表示消息鉴别码缓冲区的最大长度，返回消息鉴别码的长度。
返回值	SAR_OK:	成功。
	其他:	错误码。
备注	SKF_MacFinal 必须用于 SKF_MacUpdate 之后。	

3.1.6.42 关闭密码对象句柄

函数原型	ULONG DEVAPI SKF_CloseHandle(HANDLE hHandle)	
功能描述	关闭会话密钥、密码杂凑对象、消息鉴别码对象、ECC 密钥协商等句柄。	
参数	hHandle	[IN] 要关闭的对象句柄。
返回值	SAR_OK:	成功。
	其他:	错误码。

附录 缩略语及术语

缩略语及术语	解 释
ePass3000GM	飞天推出的采用 USB 2.0 高速接口,支持高速数据国密算法加解密,提供 CSP 和国密接口,是电子政务、各大 CA 首选的 USB Key 产品。
Token	密码设备的统称,可以是智能卡,也可以是具有密码和证书存储功能的任何硬件设备。
USB Token	具有 USB 接口的密码设备,其携带方便,操作简单。ePass3000GM 是其中一种。
CryptoAPI 接口 (简称 CAPI)	由微软公司提供的密码(cryptography)操作接口,提供设备无关的或软件实现的密码算法封装,很容易使开发者能够开发出用于数据加解密、使用数字证书的身份认证、代码签名等的 Windows 平台上的 PKI 应用程序。
智能 IC 卡及智能密码钥匙密码应用接口规范	由国家密码管理局提出并制定的基于 PKI 密码体制的智能 IC 卡及智能密码钥匙密码应用接口,描述了密码应用接口的函数、数据类型、参数的定义和设备的安全要求。